

Project Overview

Dear students you are required to develop a complete game in C++ incorporating key object-oriented programming concepts, data structures, and algorithms covered in the course. The project should demonstrate mastery of C++ features while delivering an engaging gameplay experience.

1 Sample Game Ideas

1. Dungeon Crawler RPG

- Classes: **Player**, **Monster**, **Weapon**, **Potion**
- Data Structures: Inventory (stack/queue), enemy list (linked list)
- Algorithms: Pathfinding (BFS/DFS), turn-based combat
- Libraries: **ncurses** (text UI), **Boost.Serialization** (save/load)

2. Chess/Checkers

- Classes: **Board**, **Piece** (with derived classes for each piece)
- Algorithms: Move validation (recursion), minimax AI
- OOP: Polymorphism for piece movements
- Libraries: **SFML** (graphics), **STL** for board representation

3. Tower Defense Game

- Classes: **Tower**, **Enemy**, **Projectile**
- Data Structures: Priority queue for targeting, vector for path points
- Algorithms: Wave spawning, collision detection
- Libraries: **SDL2** or **SFML** for graphics

4. Sudoku Solver with GUI

- Classes: **Board**, **Cell**
- Algorithms: Backtracking solver (recursion)
- OOP: Operator overloading for board operations
- Libraries: **Qt** or **ImGui** for interface

5. Pac-Man Clone

- Classes: **Pacman**, **Ghost**, **Maze**
- Data Structures: Queue for ghost AI, 2D array for maze
- Algorithms: Pathfinding (A* or Dijkstra)
- Libraries: **SFML** or **raylib** for graphics

6. Battleship Game

- Classes: **Player**, **Board**, **Ship**
- OOP: Template class for different board sizes
- Exception Handling: Invalid moves
- Libraries: **STL** for board representation

7. Text-Based Adventure

- Classes: **Room**, **Item**, **Player**
- Data Structures: Graph for world map, stack for inventory
- File I/O: Save/load game state
- Libraries: **ncurses** for interface

8. Snake Game

- Classes: **Snake**, **Board**, **Food**
- Data Structures: Linked list for snake body
- Algorithms: Collision detection
- Libraries: **conio.h** (Windows) or **ncurses** (Linux)

9. Blackjack (21)

- Classes: **Deck**, **Card**, **Player**
- Algorithms: Shuffling (Fisher-Yates), scoring
- OOP: Inheritance for different player types
- Libraries: **STL vector** for deck

10. Space Invaders

- Classes: **Spaceship**, **Alien**, **Bullet**
- Data Structures: Vector for aliens, queue for bullets
- OOP: Polymorphism for different alien types
- Libraries: **SFML** or **SDL2** for graphics

11. AI Neural Network Simulator

- Classes: **Neuron**, **Layer**, **Network**, **TrainingData**
- Data Structures: Graph for neural connections, matrix for weights
- Algorithms: Backpropagation, gradient descent, activation functions
- OOP: Template classes for different network architectures
- Libraries: **Eigen** for matrix operations, **SFML** for visualization

12. Procedural Planet Generator

- Classes: **Planet**, **Biome**, **PerlinNoise**, **TerrainChunk**
- Data Structures: Octree for terrain chunks, hash map for biome data
- Algorithms: Perlin/Simplex noise generation, LOD system
- OOP: Strategy pattern for different generation algorithms
- Libraries: **OpenGL**, **glm** for 3D graphics

13. Blockchain Cryptocurrency Simulator

- Classes: **Block**, **Transaction**, **Wallet**, **Chain**
- Data Structures: Linked list for blockchain, map for wallet balances
- Algorithms: SHA-256 hashing, proof-of-work, transaction validation
- OOP: Observer pattern for network updates
- Libraries: **OpenSSL** for cryptography, **ncurses** for UI

14. Quantum Computing Simulator

- Classes: **Qubit**, **Gate**, **Circuit**, **Measurement**
- Data Structures: Complex number matrices, tensor networks
- Algorithms: Quantum Fourier transform, entanglement simulation
- OOP: Factory pattern for gate creation, command pattern for circuits
- Libraries: **Armadillo** for linear algebra, **ImGui** for interface

15. Genetic Algorithm Ecosystem

- Classes: **Organism**, **Gene**, **Environment**, **Species**
- Data Structures: Priority queue for fitness selection, tree for phylogeny
- Algorithms: Crossover, mutation, natural selection simulation
- OOP: Composite pattern for complex organisms
- Libraries: **SFML** for visualization, **Boost.Random**

16. Real-time Strategy AI Engine

- Classes: **Unit**, **Base**, **Resource**, **AICommander**
- Data Structures: Spatial hash grid for unit queries, behavior trees
- Algorithms: A* pathfinding, resource allocation, fog of war
- OOP: State pattern for unit behaviors, component entity system
- Libraries: **SDL2**, **entt** for entity-component system

17. Physics Engine with Soft Bodies

- Classes: **RigidBody**, **Spring**, **Constraint**, **World**
- Data Structures: Spatial partitioning grid, contact manifolds

- Algorithms: Verlet integration, collision detection (SAT/GJK)
- OOP: Visitor pattern for collision resolution
- Libraries: **OpenGL**, custom linear algebra implementation

18. Procedural Music Composer

- Classes: **Note**, **Chord**, **Scale**, **Pattern**
- Data Structures: Markov chains for transitions, graph for chord progressions
- Algorithms: Markov chain generation, MIDI synthesis, rhythm generation
- OOP: Builder pattern for song construction
- Libraries: **PortAudio** for audio, **rtmidi** for MIDI

19. Multi-agent Traffic Simulation

- Classes: **Vehicle**, **Road**, **Intersection**, **TrafficLight**
- Data Structures: Graph for road network, priority queue for events
- Algorithms: Car-following models, lane changing, intersection control
- OOP: Factory pattern for different vehicle types
- Libraries: **SFML** for visualization, **Boost.Asio** for networking

20. AR/VR Museum Tour (Desktop Simulator)

- Classes: **Artifact**, **Gallery**, **Visitor**, **Guide**
- Data Structures: Scene graph for 3D objects, k-d tree for spatial queries
- Algorithms: Ray casting, object interaction, narrative branching
- OOP: Visitor pattern for interaction handlers
- Libraries: **OpenGL**, **OpenXR** (simulated), **Assimp** for 3D models

2 Technical Requirements

2.1 Core OOP Concepts (Must Implement)

- **Classes & Objects**: All game entities must be proper classes
- **Encapsulation**: Proper use of **private/public** with getters/setters
- **Constructors/Destructors**: Including copy constructors where needed
- **Operator Overloading**: For game logic (e.g., **player1 == player2**)
- **Inheritance & Polymorphism**: Class hierarchies with virtual functions
- **Abstract Classes**: At least one pure virtual function interface

2.2 Data Structures & Algorithms (Choose at least 3)

- **Dynamic Arrays/Linked Lists:** For inventories, enemy lists, etc.
- **Stack/Queue:** For undo/redo, pathfinding, or turn management
- **Recursion:** For puzzles, maze generation, or backtracking
- **Sorting Algorithms:** For leaderboards or item organization
- **Priority Queues:** For event scheduling or damage systems

2.3 Advanced C++ Features (Choose at least 2)

- **Templates:** Generic containers or functions
- **Exception Handling:** Robust input validation and error recovery
- **STL Containers:** Proper use of `vector`, `map`, etc.
- **Lambdas:** For event callbacks or AI behavior
- **Friend Classes:** For controlled access between classes

3 Course Topic Applications

Topic	Game Implementation Example
Classes/Objects	All game entities (Player, Enemy, Item) as classes
Encapsulation	Private member variables with public getters/setters
Operator Overloading	<code>operator+</code> for combining items, <code>operator<<</code> for output
Inheritance	<code>Enemy</code> → <code>FlyingEnemy</code> , <code>GroundEnemy</code> hierarchy
Abstract Classes	<code>Renderable</code> interface with <code>render()</code> pure virtual function
Templates	<code>Inventory<Item></code> container class
STL Containers	<code>std::vector</code> for bullets, <code>std::map</code> for game settings
Recursion	Maze generation using recursive division
Sorting	MergeSort for high score tables
Exception Handling	Invalid move exceptions in board games

4 Recommended Libraries

- **Text UI:** ncurses (pdcurses for Windows)
- **Serialization:** Boost.Serialization, JSON for Modern C++
- **Utility:** STL, Boost (for smart pointers, etc.)
- **Graphics:** SFML, SDL2, raylib, OpenGL **BGFX** (cross-platform), **Magnum** (C++11/14 focused)
- **Physics:** Box2D (2D), Bullet (3D, but can be complex)

- **UI:** Dear ImGui (immediate mode GUI)
 - **Audio:** SoLoud (simple, portable audio engine)
 - **Serialization:** cereal (header-only C++11 serialization)
 - **ECS:** entt (modern entity-component-system library)
 - **Testing:** Catch2 or doctest for unit testing
- <https://github.com/codecrafters-io/build-your-own-x>

Submission Requirements

- Complete source code (properly structured .h/.cpp files)
- Makefile/CMakeLists.txt for building
- Documentation (README file)
- 5-10 page report explaining design decisions
- Demonstration (executable or video)

***** Best Of Luck*****